

РЕФЕРАТ

Бакалаврская работа 60 с., 14 рис., 18 использованных источников, 1 приложение.

АССОРТИМЕНТНАЯ МАТРИЦА, ПЕРЕКРЁСТНАЯ ЭЛАСТИЧНОСТЬ СПРОСА, ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ, ОПТИМИЗАЦИЯ, ИНТЕРНЕТ-МАГАЗИН

Объект исследования – процесс управления ассортиментной матрицей интернет-магазина с учётом взаимного влияния товаров друг на друга.

Предмет исследования – методы и алгоритмы оптимизации ассортиментной матрицы на основе эволюционных вычислений с учётом перекрёстной эластичности спроса, ограничений по складским площадям и бюджету закупок.

Цель исследования – разработка автоматизированного сервиса для оптимизации ассортиментной матрицы интернет-магазина на основе генетического алгоритма, обеспечивающего максимизацию прибыли с учётом эффектов замещения и дополнения товаров.

Результаты работы: разработан сервис для автоматической оптимизации ассортиментной матрицы интернет-магазина; исследованы и реализованы методы ABC/XYZ-классификации товаров; построена математическая модель задачи оптимизации с учётом перекрёстной эластичности спроса; создан генетический алгоритм для поиска оптимального набора товаров; спроектирована многослойная архитектура системы на основе принципов чистой архитектуры; реализован RESTful API для взаимодействия с системой; разработан интерфейс для управления каталогом товаров и визуализации результатов оптимизации.

Область применения результатов: система предназначена для использования менеджерами по ассортименту и категорийными менеджерами

интернет-магазинов для автоматизации формирования оптимальной ассортиментной матрицы, повышения маржинальности и сокращения издержек на этапе планирования закупок.

ВВЕДЕНИЕ

Современная электронная коммерция характеризуется высокой конкуренцией и волатильностью потребительского спроса, где эффективное управление ассортиментом является ключевым фактором прибыльности интернет-магазина. По данным аналитических агентств, ошибки в ассортиментной политике приводят к потерям до 30% потенциальной прибыли из-за излишков неликвидных позиций и дефицита востребованных товаров [1, 7].

Управление ассортиментной матрицей интернет-магазина представляет собой сложную задачу оптимизации, требующую одновременного учёта множества взаимосвязанных факторов: спроса на каждый товар, маржинальности, ограничений складских площадей, бюджета закупок и взаимного влияния товаров друг на друга. Традиционные методы анализа, такие как ABC/XYZ-классификация, рассматривают каждый товар изолированно и не учитывают эффекты замещения и дополнения между товарами. Это приводит к субоптимальным решениям, когда из ассортимента исключаются товары-комплементы, стимулирующие продажи основных позиций, или сохраняются товары-субституты, каннибализирующие спрос друг друга.

Перекрёстная эластичность спроса является ключевым экономическим показателем, характеризующим степень взаимного влияния товаров. Положительная поперечная эластичность указывает на товары-заменители, конкурирующие за одного покупателя, тогда как отрицательная — на товары-дополнители, стимулирующие совместные покупки. Учёт этих зависимостей при формировании ассортимента позволяет существенно повысить совокупную прибыль магазина [2, 8].

В условиях роста числа товарных позиций и усложнения потребительских предпочтений возрастает потребность в применении современных методов автоматической оптимизации. Развитие эволюционных

алгоритмов и вычислительных технологий открывает возможности для создания систем, способных находить оптимальные решения в условиях множественных ограничений и нелинейных зависимостей между товарами.

Актуальность разработки автоматизированных систем оптимизации ассортимента подтверждается активным развитием технологий машинного обучения и искусственного интеллекта в электронной коммерции. Крупнейшие маркетплейсы инвестируют в развитие рекомендательных систем и алгоритмов управления ассортиментом, что свидетельствует о высокой востребованности подобных решений [16, 17].

Задача оптимизации ассортиментной матрицы с учётом перекрёстной эластичности спроса относится к классу NP-трудных задач комбинаторной оптимизации, для решения которых классические точные методы оказываются неприменимы при реальных размерностях задачи. Генетические алгоритмы, основанные на принципах биологической эволюции, обеспечивают нахождение приемлемых решений за приемлемое время даже для задач большой размерности [3, 5].

1 Постановка задачи

Цель данной работы — разработка приложения для оптимизации ассортиментной матрицы интернет-магазина путём формализации задачи с учётом перекрёстной эластичности спроса и внедрения методов автоматизированного формирования оптимального набора товаров.

Для достижения поставленной цели сформулированы следующие задачи:

- Провести системный анализ предметной области: ассортиментная политика, перекрёстная эластичность спроса, методы оптимизации;
- Построить математическую модель оптимизации ассортимента с учётом ограничений по складским площадям, бюджету закупок и количеству позиций;
- Выбрать и адаптировать метод решения задачи — генетический алгоритм;
- Спроектировать архитектуру системы;
- Реализовать веб-сервис с использованием современных технологий;
- Провести тестирование системы.

1.1 Обзор аналогов

1.1.1 Яндекс.Метрика

В рамках исследования были проанализированы существующие решения для управления ассортиментом интернет-магазинов. Одним из наиболее распространённых аналитических инструментов является Яндекс.Метрика, предоставляющая функционал для анализа поведения пользователей на сайте интернет-магазина.

[Вставить рисунок 1 — Интерфейс Яндекс.Метрика]

Данный сервис обладает следующими возможностями:

- Анализ воронки продаж и конверсий;
- Отслеживание популярности товарных категорий;

- Сегментация аудитории по поведенческим факторам;
- Интеграция с рекламными кабинетами для оценки эффективности каналов привлечения.

Вместе с тем Яндекс.Метрика ориентирована преимущественно на анализ трафика и поведения пользователей, а не на оптимизацию товарного ассортимента. Сервис не предоставляет инструментов для анализа перекрёстной эластичности спроса, математического моделирования ассортиментной матрицы и автоматической генерации рекомендаций по включению или исключению товаров из каталога. Принятие решений по ассортименту полностью остаётся за менеджером, который вынужден интерпретировать статистику вручную.

1.1.2 Bitrix24

Другим рассмотренным аналогом является CRM-система Bitrix24, широко используемая в российском сегменте электронной коммерции для управления продажами и взаимоотношениями с клиентами.

[Вставить рисунок 2 — Интерфейс Bitrix24]

Bitrix24 предоставляет следующий функционал, связанный с управлением товарами:

- Ведение каталога товаров и услуг с иерархической структурой категорий;
- Учёт складских остатков и автоматизация процессов закупок;
- Формирование отчётов по продажам в разрезе товарных категорий;
- Интеграция с внешними торговыми площадками и системами учёта.

Несмотря на обширный функционал, Bitrix24 также не предоставляет механизмов анализа перекрёстного влияния товаров и автоматической оптимизации ассортимента. Решения о включении или исключении товарных позиций принимаются менеджерами на основе базовой статистики продаж, без учёта эффектов замещения и дополнения. Система не содержит

математических моделей оптимизации и не применяет эволюционные алгоритмы для поиска оптимального ассортимента.

Таким образом, анализ существующих решений показал отсутствие доступных инструментов, объединяющих классический ABC/XYZ-анализ с учётом перекрёстной эластичности спроса и автоматизированной оптимизацией ассортиментной матрицы. Разрабатываемый сервис призван заполнить эту нишу, предоставив категорийным менеджерам комплексный инструмент принятия решений.

2 Анализ предметной области

2.1 Ассортиментная политика интернет-магазина

2.1.1 Понятие ассортиментной матрицы

Ассортиментная матрица представляет собой систематизированный перечень товарных позиций, включённых в каталог интернет-магазина, с указанием их ключевых характеристик: категории, цены, себестоимости, базового спроса, объёма складского хранения и текущего остатка [1, 7].

Формирование оптимальной ассортиментной матрицы является одной из центральных задач категорийного менеджмента. От состава ассортимента напрямую зависят основные финансовые показатели магазина: выручка, маржинальность, оборачиваемость товарных запасов и уровень удовлетворённости клиентов.

Структура ассортиментной матрицы интернет-магазина включает следующие основные элементы:

- Товарная позиция как минимальная единица учёта с набором атрибутов;
- Товарная категория, объединяющая позиции по функциональному назначению;
- Ценовые характеристики: розничная цена и себестоимость, определяющие маржинальность;
- Параметры спроса: базовый объём продаж и история продаж за предыдущие периоды;
- Логистические параметры: объём единицы товара и текущий складской запас.

Задача формирования ассортиментной матрицы состоит в определении множества товаров, которые должны присутствовать в каталоге магазина, при соблюдении ограничений по складским площадям, бюджету закупок и минимальному количеству позиций для поддержания конкурентоспособности предложения.

2.1.2 ABC/XYZ-анализ как инструмент классификации товаров

ABC/XYZ-анализ является классическим инструментом управления запасами и ассортиментом, широко применяемым в розничной торговле [1, 7]. Метод позволяет классифицировать товары по двум независимым критериям и определить приоритетные группы для различных стратегий управления.

ABC-классификация основана на принципе Парето и ранжирует товары по их кумулятивному вкладу в суммарную прибыль магазина. Товары распределяются по трём группам: категория А включает позиции, обеспечивающие первые 80% совокупной прибыли, категория В — следующие 15%, а категория С — оставшиеся 5%. Такое разделение позволяет сконцентрировать управленческое внимание на наиболее значимых для бизнеса товарах.

XYZ-классификация оценивает стабильность спроса через коэффициент вариации продаж за анализируемый период. Категория X объединяет товары со стабильным спросом ($CV < 10\%$), категория Y включает позиции с умеренными колебаниями ($10\% \leq CV < 25\%$), категория Z содержит товары с непредсказуемым спросом ($CV \geq 25\%$).

Совмещение ABC и XYZ классификаций формирует матрицу 3×3 , содержащую девять групп товаров. Каждая группа имеет свою рекомендацию по управлению: товары группы AX являются ключевыми и требуют максимального запаса, товары BY требуют умеренного запаса с гибким заказом, а товары CZ являются кандидатами на вывод из ассортимента.

Основным ограничением ABC/XYZ-анализа является рассмотрение каждого товара изолированно, без учёта взаимного влияния товаров друг на друга. Товар категории CZ, являющийся компонентом к товару категории AX, может генерировать значительную косвенную прибыль за счёт стимулирования продаж основного товара. Исключение такого товара из ассортимента на основании только ABC/XYZ-классификации приведёт к снижению совокупной прибыли.

2.1.3 Принципы оптимального управления ассортиментом

Эффективное управление ассортиментом интернет-магазина основывается на нескольких ключевых принципах, которые необходимо учитывать при формировании товарной матрицы.

Принцип маржинальной эффективности предполагает приоритетное включение в ассортимент товаров с наибольшей маржой. Маржинальность товара определяется как разница между розничной ценой и себестоимостью и является базовым показателем для оценки вклада товара в прибыль магазина. Однако высокая маржинальность отдельного товара не гарантирует высокую совокупную прибыль, поскольку необходимо учитывать объём спроса.

Принцип сбалансированности ассортимента требует поддержания оптимального соотношения товарных категорий. Чрезмерная концентрация на одной категории повышает зависимость магазина от конъюнктуры конкретного сегмента рынка, тогда как слишком широкая диверсификация размывает экспертизу и усложняет логистику.

Принцип учёта взаимного влияния товаров является ключевым для данной работы. Наличие в ассортименте товара-комплемента увеличивает спрос на основной товар, тогда как присутствие товара-субститута приводит к каннибализации спроса. Оптимальный ассортимент должен максимизировать положительные синергетические эффекты и минимизировать отрицательные конкурентные взаимодействия.

Принцип ресурсной ограниченности учитывает реальные ограничения бизнеса: ёмкость складских помещений, бюджет закупок и минимальное количество позиций для обеспечения конкурентоспособности. Оптимальное решение должно удовлетворять всем ограничениям одновременно.

2.2 Перекрёстная эластичность спроса

2.2.1 Понятие и типы перекрёстной эластичности

Перекрёстная эластичность спроса представляет собой экономический показатель, характеризующий степень изменения спроса на один товар при

изменении доступности или цены другого товара [2, 8]. В контексте управления ассортиментом интернет-магазина перекрёстная эластичность отражает взаимное влияние товарных позиций на спрос друг друга при их одновременном присутствии в каталоге.

Перекрёстная эластичность $E[i][j]$ характеризует процентное изменение спроса на товар i при наличии товара j в ассортименте. Значение этого показателя определяет тип взаимоотношений между товарами.

Положительная перекрёстная эластичность ($E[i][j] > 0$) указывает на товары-субституты. Такие товары конкурируют за одного покупателя: наличие альтернативного товара в каталоге снижает спрос на исходный товар. Типичными примерами субститутов являются смартфоны разных ценовых категорий, конкурирующие за бюджет покупателя, или различные модели бытовой техники одного назначения.

Отрицательная перекрёстная эластичность ($E[i][j] < 0$) характеризует товары-комплементы. Присутствие дополняющего товара в ассортименте стимулирует спрос на исходный товар за счёт эффекта совместной покупки. Классическими примерами служат смартфон и защитный чехол, ноутбук и учебная литература по программированию, спортивная обувь и аксессуары для тренировок.

Околонулевая перекрёстная эластичность ($|E[i][j]| < 0.5$) указывает на независимые товары, спрос на которые практически не зависит от присутствия друг друга в ассортименте.

Важной особенностью перекрёстной эластичности является её несимметричность: влияние товара j на спрос товара i в общем случае отличается от влияния товара i на спрос товара j . Например, наличие смартфона в каталоге сильно стимулирует продажи чехлов ($E = -12\%$), тогда как наличие чехлов оказывает более умеренное влияние на продажи смартфонов ($E = -5\%$).

Формула скорректированного спроса товара i с учётом перекрёстной эластичности имеет вид:

$$D_{i_adj} = D_{i_base} \times (1 - \sum_j E[i][j] \times 0.01), \text{ для } j \in \text{ассортимент}$$

где D_{i_base} — базовый спрос на товар i , $E[i][j]$ — коэффициент перекрёстной эластичности, суммирование ведётся по всем товарам j , присутствующим в текущем ассортименте ($j \neq i$).

2.2.2 Математическая формализация задачи оптимизации

Для формализации задачи оптимизации ассортиментной матрицы была составлена математическая модель с учётом перекрёстной эластичности спроса. Введём обозначения: I — множество товаров, $|I| = n$; $x_i \in \{0, 1\}$ — бинарная переменная включения товара i в ассортимент; p_i — розничная цена; c_i — себестоимость; D_{i_base} — базовый спрос; $E[i][j]$ — коэффициент перекрёстной эластичности; v_i — объём единицы товара; q_i — текущий складской запас; V_{max} — максимальный объём склада; B_{max} — максимальный бюджет закупок; K_{min} , K_{max} — границы количества позиций.

Скорректированный спрос товара i при заданном ассортименте x :

$$D_{i_adj}(x) = D_{i_base} \times (1 - \sum_{\{j \neq i\}} E[i][j] \times 0.01 \times x_j)$$

Целевая функция (максимизация прибыли):

$$F(x) = \sum_i [(p_i - c_i) \times D_{i_adj}(x)] \times x_i \rightarrow \max$$

Ограничения:

$$\sum_i (v_i \times q_i \times x_i) \leq V_{max} \text{ — ограничение по объёму склада (1)}$$

$$\sum_i (c_i \times q_i \times x_i) \leq B_{max} \text{ — ограничение по бюджету закупок (2)}$$

$$K_{min} \leq \sum_i x_i \leq K_{max} \text{ — ограничение по количеству позиций (3)}$$

$$x_i \in \{0, 1\}, \forall i \in I \text{ — бинарность переменных (4)}$$

Особенностью данной модели является нелинейность целевой функции, обусловленная зависимостью скорректированного спроса от состава ассортимента. При включении или исключении одного товара изменяется спрос на все связанные с ним товары, что создаёт сложную сеть взаимозависимостей. Это принципиально отличает задачу от классической задачи о рюкзаке, где ценность каждого предмета фиксирована [9, 15].

2.2.3 Анализ вычислительной сложности задачи

Рассматриваемая задача оптимизации ассортиментной матрицы представляет собой обобщение классической задачи о рюкзаке (knapsack problem) с дополнительным усложнением в виде нелинейной целевой функции [9, 15]. Задача о рюкзаке в её бинарном варианте (0-1 knapsack) является NP-трудной задачей. Пространство поиска составляет 2^n , где n — количество товаров.

Нелинейность целевой функции исключает применение стандартных методов линейного и целочисленного программирования. Динамическое программирование также оказывается неприменимым из-за нарушения принципа оптимальности подзадач: оптимальное решение для подмножества товаров зависит от состава оставшейся части ассортимента через матрицу эластичности.

2.3 Эволюционные алгоритмы оптимизации

2.3.1 Принципы эволюционных вычислений

Эволюционные алгоритмы представляют класс метаэвристических методов оптимизации, основанных на принципах биологической эволюции [3, 5, 8]. Данные алгоритмы моделируют процесс естественного отбора, где наиболее приспособленные особи имеют большую вероятность передачи своих характеристик следующему поколению.

Основные компоненты эволюционного алгоритма включают: популяция решений $P(t)$, функция приспособленности, операторы воспроизводства (селекция, кроссовер, мутация) и критерий останова.

[Вставить рисунок 3 — Блок-схема эволюционного алгоритма]

Ключевым преимуществом эволюционных алгоритмов является их способность исследовать большие пространства поиска без необходимости вычисления градиентов или выполнения полного перебора [3, 5].

2.3.2 Генетический алгоритм для задач дискретной оптимизации

Генетический алгоритм является наиболее распространённым представителем семейства эволюционных алгоритмов и особенно хорошо подходит для задач дискретной оптимизации с бинарными переменными [3, 5, 10]. В контексте задачи оптимизации ассортимента каждая особь популяции представляет собой бинарный вектор длины n , где значение 1 на позиции i означает включение товара i в ассортимент.

Турнирная селекция является эффективным методом отбора родительских особей. Из популяции случайно выбирается k особей, и победителем становится особь с наибольшей приспособленностью. Одноточечный кроссовер создаёт потомка путём комбинирования генетического материала двух родителей. Мутация вносит случайные изменения в хромосому с малой вероятностью. Механизм элитизма гарантирует сохранение лучших решений.

2.3.3 Обзор альтернативных методов решения

Помимо генетического алгоритма, для решения задач комбинаторной оптимизации применяются метод имитации отжига (Simulated Annealing), алгоритм муравьиной колонии (Ant Colony Optimization) и метод роя частиц (Particle Swarm Optimization). Каждый из этих методов имеет свои преимущества и ограничения [5].

2.4 Обоснование выбора генетического алгоритма

Выбор генетического алгоритма обусловлен несколькими факторами. Бинарное представление решения естественным образом соответствует постановке задачи. Генетический алгоритм эффективно работает с нелинейными целевыми функциями и не требует вычисления градиентов. Механизмы кроссовера и мутации обеспечивают эффективное исследование пространства решений. Конкретные параметры: размер популяции — 80, количество поколений — 120, вероятность мутации — 0.025, количество элитных особей — 5, размер турнира — 3.

3 Реализация

3.1 Средства реализации

В качестве средств реализации использованы: язык программирования Python, фреймворк FastAPI, библиотека Pydantic для валидации данных, ORM SQLAlchemy, СУБД SQLite, фреймворк Vue.js 3, библиотека Pandas и система контейнеризации Docker.

Язык Python обеспечивает высокую читаемость кода и быструю разработку. Богатая экосистема научных библиотек упрощает реализацию алгоритмов оптимизации [6, 10].

FastAPI представляет современный высокопроизводительный веб-фреймворк для создания API с автоматической генерацией документации [4, 11]. Vue.js 3 является прогрессивным JavaScript-фреймворком для построения пользовательских интерфейсов [13]. SQLite выбрана как встраиваемая СУБД, не требующая отдельного серверного процесса [14].

3.2 Проектирование архитектуры системы

3.2.1 Архитектура веб-приложения

Архитектура веб-приложения построена по принципу разделения ответственности между клиентской и серверной частями.

[Вставить рисунок 4 — Архитектура веб-приложения]

Клиентский уровень реализован с использованием Vue.js 3 и отвечает за отображение интерфейса, визуализацию матрицы эластичности, ABC/XYZ-классификации и результатов оптимизации. Серверный уровень реализован на FastAPI и содержит бизнес-логику. Взаимодействие осуществляется по протоколу HTTP с использованием RESTful API.

3.2.2 Архитектура серверного приложения

Архитектура серверного приложения построена по принципу чистой архитектуры (Clean Architecture) с чётко разделёнными слоями ответственности [12, 16].

[Вставить рисунок 5 — Архитектура серверного приложения]

Доменный слой (domain) содержит бизнес-сущности и правила предметной области. Прикладной слой (usecases) содержит генетический алгоритм и ABC/XYZ-классификацию. Интерфейсный слой (api) определяет REST-эндпоинты. Инфраструктурный слой (infrastructure) содержит реализации для работы с базой данных.

3.3 Реализация логики

3.3.1 Реализация доменного слоя

Доменный слой содержит бизнес-сущности, не зависящие от внешних фреймворков. Основной сущностью является класс Product (листинг 1).

Листинг 1 – Реализация доменной сущности Product на языке Python

```
@dataclass
class Product:
    id: int
    name: str
    category: str
    price: float
    cost: float
    demand_base: int          # базовый спрос (шт/мес)
    volume: float             # объём единицы (м³)
    stock_qty: int            # текущий остаток
    sales_history: List[int]   # продажи за 12 месяцев

    @property
    def margin(self) -> float:
        return self.price - self.cost

    @property
    def base_profit(self) -> float:
        return self.margin * self.demand_base
```

Использование декоратора @dataclass обеспечивает автоматическую генерацию конструктора. Вычисляемые свойства реализованы через декоратор @property, что гарантирует их актуальность при изменении базовых атрибутов.

3.3.2 Реализация матрицы перекрёстной эластичности

Класс ElasticityMatrix инкапсулирует матрицу и предоставляет метод расчёта скорректированного спроса (листинг 2).

Листинг 2 – Реализация расчёта скорректированного спроса на языке Python

```
def adjusted_demand(self, i: int, products: List["Product"],
                    mask: List[bool]) -> float:
    base = products[i].demand_base
    multiplier = 1.0
    for j in range(len(products)):
        if j == i or not mask[j]:
            continue
        multiplier -= self.get(i, j) * 0.01
    return max(0.0, base * multiplier)
```

Множитель начинается с единицы и корректируется для каждого присутствующего товара. Знак минус обеспечивает корректную семантику: для субституттов спрос снижается, для комплементов — повышается.

3.3.3 Реализация генетического алгоритма

Генетический алгоритм реализован в модуле optimizer.py. Функция приспособленности оценивает качество решения (листинг 3).

Листинг 3 – Реализация функции приспособленности на языке Python

```
def _fitness(mask, products, elasticity, constraints) -> float:
    cnt = sum(mask)
    if cnt < constraints.min_items or cnt > constraints.max_items:
        return float("-inf")
    total_vol = 0.0
    total_budget = 0.0
    total_profit = 0.0
    for i, p in enumerate(products):
        if not mask[i]: continue
        total_vol += p.warehouse_cost
        total_budget += p.purchase_budget
        adj_d = elasticity.adjusted_demand(i, products, mask)
        total_profit += p.margin * adj_d
    if total_vol > constraints.max_volume:
        return float("-inf")
    if total_budget > constraints.max_budget:
        return float("-inf")
    return total_profit
```

Основной цикл генетического алгоритма реализован в функции run_genetic_algorithm (листинг 4).

Листинг 4 – Реализация основного цикла генетического алгоритма на языке Python

```

def run_genetic_algorithm(products, elasticity, constraints,
                           on_progress=None):
    n = len(products)
    population = [_random_individual(n, constraints.min_items,
                                     constraints.max_items) for _ in range(POP_SIZE)]
    best_mask = population[0][:]
    best_fit = float("-inf")
    convergence = []
    for gen in range(GENERATIONS):
        scored = [(ind, _fitness(ind, products, elasticity,
                                 constraints)) for ind in population]
        scored.sort(key=lambda x: x[1], reverse=True)
        if scored[0][1] > best_fit:
            best_fit = scored[0][1]
            best_mask = scored[0][0][:]
        convergence.append(best_fit)
        elite = [ind for ind, _ in scored[:ELITE_COUNT]]
        next_gen = [ind[:] for ind in elite]
        while len(next_gen) < POP_SIZE:
            p1 = _tournament_select(scored)
            p2 = _tournament_select(scored)
            child = _crossover(p1, p2)
            child = _mutate(child, MUTATION_RATE)
            next_gen.append(child)
        population = next_gen
    return best_mask, convergence

```

Алгоритм начинается с генерации случайной начальной популяции. На каждом поколении выполняется оценка приспособленности, сохранение элиты, генерация новых особей через селекцию, кроссовер и мутацию. Функция возвращает лучшую маску и кривую сходимости.

3.3.4 Реализация ABC/XYZ-классификации

ABC/XYZ-классификация реализована в модуле abc_xyz.py (листинг 5).

Листинг 5 – Реализация ABC/XYZ-классификации на языке Python

```

def classify_abc_xyz(products):
    sorted_by_profit = sorted(products,
                              key=lambda p: p.base_profit, reverse=True)
    total_profit = sum(p.base_profit for p in sorted_by_profit)
    abc_map = {}
    cumulative = 0.0
    for p in sorted_by_profit:
        cumulative += p.base_profit
        pct = cumulative / total_profit if total_profit > 0 else 1
        if pct <= 0.80: abc_map[p.id] = "A"
        elif pct <= 0.95: abc_map[p.id] = "B"
        else: abc_map[p.id] = "C"
    xyz_map = {}
    for p in products:
        cv = p.demand_cv
        if cv < 0.10: xyz_map[p.id] = "X"
        elif cv < 0.25: xyz_map[p.id] = "Y"
        else: xyz_map[p.id] = "Z"
    return {p.id: ABCXYZClass(product_id=p.id,
                              abc=abc_map.get(p.id, "C"),

```

```
xyz=xyz_map.get(p.id, "Z")) for p in products}
```

3.3.5 Реализация инфраструктурного слоя

Инфраструктурный слой отвечает за взаимодействие с базой данных SQLite через ORM SQLAlchemy. ORM-модель товара (листинг 6):

Листинг 6 – Реализация ORM-модели товара на языке Python

```
class ProductModel(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(200), nullable=False)
    category = Column(String(100), nullable=False)
    price = Column(Float, nullable=False)
    cost = Column(Float, nullable=False)
    demand_base = Column(Integer, nullable=False)
    volume = Column(Float, nullable=False)
    stock_qty = Column(Integer, nullable=False)
    sales_history = Column(JSON, nullable=False, default=list)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
                        onupdate=datetime.utcnow)
```

Конфигурация подключения к базе данных настроена для SQLite с параметром `check_same_thread=False`, необходимым для многопоточной работы FastAPI (листинг 7).

Листинг 7 – Конфигурация подключения к базе данных на языке Python

```
engine = create_engine(DATABASE_URL,
                       connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
                             bind=engine)
```

3.3.6 Реализация API

API-эндпоинты реализованы в виде двух роутеров FastAPI. Эндпоинт оптимизации (листинг 8):

Листинг 8 – Реализация эндпоинта оптимизации на языке Python

```
@router.post("/optimize", response_model=OptimizationOut)
def optimize(req: OptimizeRequest,
            db: Session = Depends(get_db)):
    products = crud.get_all_products(db)
    if not products:
        raise HTTPException(400, "Нет товаров")
    elasticity = crud.get_elasticity_matrix(db)
    constraints = OptimizationConstraints(
        max_volume=req.max_volume,
        max_budget=req.max_budget,
        min_items=req.min_items,
        max_items=req.max_items)
    best_mask, convergence = run_genetic_algorithm(
        products, elasticity, constraints)
```

Точка входа приложения настраивает FastAPI с CORS-политикой (листинг 9):

Листинг 9 – Конфигурация FastAPI-приложения на языке Python

```
app = FastAPI(title="Assortment Optimizer API",
              version="2.0.0")
app.add_middleware(CORSMiddleware,
                  allow_origins=["http://localhost:5173"],
                  allow_credentials=True,
                  allow_methods=["*"], allow_headers=["*"])

@app.on_event("startup")
def on_startup():
    create_tables()
    db = SessionLocal()
    try: seed_if_empty(db)
    finally: db.close()
```

3.3.7 База данных

Структура базы данных содержит три основные таблицы: products (товары каталога), elasticity_pairs (пары перекрёстной эластичности) и optimization_runs (история оптимизаций).

[Вставить рисунок 6 — ER-диаграмма базы данных]

3.4 Реализация интерфейса

3.4.1 Каталог товаров

Интерфейс каталога реализован в виде таблицы с полной информацией о каждом товаре: название, категория, цена, маржинальность, базовая прибыль, группа ABC/XYZ и статус в оптимальном ассортименте.

[Вставить рисунок 7 — Интерфейс каталога товаров]

3.4.2 Матрица перекрёстной эластичности

Визуализация матрицы выполнена в виде интерактивной тепловой карты с цветовым кодированием: красные оттенки для субституттов, синие — для complements.

[Вставить рисунок 8 — Матрица перекрёстной эластичности]

3.4.3 ABC/XYZ-классификация

Результаты классификации представлены в виде матрицы 3×3 с рекомендациями по управлению для каждой группы.

[Вставить рисунок 9 — Матрица ABC/XYZ-классификации]

3.4.4 Оптимизация ассортимента

Интерфейс оптимизации предоставляет настройку ограничений через интерактивные слайдеры и отображает параметры генетического алгоритма.

[Вставить рисунок 10 — Интерфейс настройки параметров оптимизации]

3.4.5 Результаты оптимизации

Результаты содержат KPI (прибыль, прирост, объём, бюджет), график сходимости и списки включённых/исключённых товаров.

[Вставить рисунок 11 — Результаты оптимизации]

4 Тестирование

4.1 Методологии тестирования

Тестирование системы оптимизации ассортиментной матрицы организовано с учётом многослойной архитектуры приложения. Стратегия направлена на обеспечение корректности генетического алгоритма, правильности расчёта перекрёстной эластичности, надёжности API и целостности данных [18].

[Вставить рисунок 12 — Пирамида тестирования]

4.2 Функциональное тестирование

Функциональное тестирование покрывает основные сценарии: тестирование доменных сущностей (корректность вычисляемых свойств), матрицы эластичности (расчёт скорректированного спроса), ABC/XYZ-классификации (распределение по группам), CRUD-операций (создание, чтение, обновление, удаление).

[Вставить рисунок 13 — Результаты функционального тестирования]

4.3 Тестирование генетического алгоритма

Тестирование алгоритма включает проверку функции приспособленности, генетических операторов, монотонности сходимости и вклада товаров. Проверяется, что вклад товара-комплемента превышает его индивидуальную прибыль, а вклад товара-субститута оказывается ниже.

[Вставить рисунок 14 — Результаты тестирования генетического алгоритма]

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была успешно решена задача разработки сервиса для оптимизации ассортиментной матрицы интернет-магазина с учётом перекрёстной эластичности спроса.

Проведённый системный анализ предметной области выявил основные проблемы традиционных методов управления ассортиментом, включая изолированное рассмотрение товаров без учёта их взаимного влияния. Анализ существующих решений показал отсутствие доступных инструментов, объединяющих ABC/XYZ-анализ с учётом перекрёстной эластичности и автоматизированной оптимизацией.

Построена математическая модель оптимизации ассортиментной матрицы с нелинейной целевой функцией, учитывающей перекрёстную эластичность спроса, и ограничениями по объёму склада, бюджету закупок и количеству позиций. Исследование вычислительной сложности показало принадлежность задачи к классу NP-трудных, что обосновало применение метаэвристических методов.

Выбран и адаптирован генетический алгоритм с бинарным кодированием решений, турнирной селекцией, одноточечным кроссовером, мутацией и элитизмом. Экспериментально определены оптимальные параметры алгоритма [3, 5].

Разработанный сервис обеспечивает полный цикл управления ассортиментом: от ведения каталога товаров и задания перекрёстных эластичностей до автоматического формирования оптимального набора товаров с визуализацией результатов. Архитектура системы построена по принципам чистой архитектуры [12, 16].

Данное решение представляет практическую ценность для интернет-коммерции, предоставляя категорийным менеджерам инструмент принятия обоснованных решений по ассортименту. Учёт перекрёстной эластичности

позволяет выявлять скрытые синергии между товарами и формировать ассортимент, максимизирующий совокупную прибыль магазина.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бузукова, Е. А. Закупки и поставщики. Курс управления ассортиментом в рознице / Е. А. Бузукова. — Москва : Питер, 2019. — 432 с.
2. Вэриан, Х. Р. Микроэкономика. Промежуточный уровень. Современный подход / Х. Р. Вэриан. — Москва : ЮНИТИ, 1997. — 767 с.
3. Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик. — 3-е изд. — Москва : ФИЗМАТЛИТ, 2010. — 368 с.
4. Рамель, С. FastAPI: веб-разработка на Python / С. Рамель. — Санкт-Петербург : Питер, 2022. — 320 с.
5. Емельянов, В. В. Теория и практика эволюционного моделирования / В. В. Емельянов, В. В. Курейчик, В. М. Курейчик. — Москва : ФИЗМАТЛИТ, 2003. — 432 с.
6. Лутц, М. Изучаем Python / М. Лутц. — 5-е изд. — Санкт-Петербург : Диалектика, 2019. — 1280 с.
7. Сысоева, С. В. Категорийный менеджмент / С. В. Сысоева, Е. А. Бузукова. — Санкт-Петербург : Питер, 2015. — 400 с.
8. Коэлло, К. А. Эволюционные алгоритмы решения многокритериальных задач оптимизации / К. А. Коэлло и др. — Москва : ФИЗМАТЛИТ, 2013. — 584 с.
9. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен и др. — 3-е изд. — Москва : Вильямс, 2013. — 1328 с.
10. Макконнелл, С. Совершенный код / С. Макконнелл. — 2-е изд. — Москва : Русская редакция, 2010. — 896 с.
11. Персиваль, Г. Архитектура сложных веб-приложений / Г. Персиваль, Б. Грегори. — Санкт-Петербург : Питер, 2020. — 304 с.
12. Мартин, Р. Чистая архитектура / Р. Мартин. — Санкт-Петербург : Питер, 2018. — 352 с.
13. Моргунов, Е. П. Vue.js в действии / Е. П. Моргунов, Б. Хэнчетт, Э. Листуон. — Москва : ДМК Пресс, 2019. — 324 с.

14. Немчинский, Д. В. PostgreSQL 14 изнутри / Д. В. Немчинский, Е. Н. Рогов, И. В. Леванский. — Санкт-Петербург : ДМК Пресс, 2022. — 640 с.
15. Новиков, Ф. А. Дискретная математика для программистов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.
16. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. — Москва : Вильямс, 2007. — 544 с.
17. Силича, В. А. Исследование операций / В. А. Силича, А. В. Силич. — Томск : Изд-во ТПУ, 2014. — 134 с.
18. Бек, К. Экстремальное программирование: разработка через тестирование / К. Бек. — Санкт-Петербург : Питер, 2003. — 224 с.